Lema del Mínimo Liso — Tratado 33×1

Proyecto 33×1 · Criba de Mínimos Lisos → candidatos GNFS Autor del concepto: Víctor Manzanares Alberola (espiradesombra) · Implementación: Lee Arbusto

Origen

El generador de gráficas del **Libro 2, "Números otra vez"**, cazaba los *puntos de inicio de secuencias lineales de restos* al dividir un número por naturales consecutivos. Ese truco, formalizado, da lugar al presente Lema.

Definiciones

Para un entero `n` fijo y naturales consecutivos `i = 2, 3, 4, ...`:

- **Resto:** `r_i = n mod i`
- **Cociente mínimo:** `q_i = (n − (n mod i)) / i` — el máximo múltiplo de `i` que no se pasa de `n`.
- **Mínimo relativo:** `i` tal que `r_i ≤ r_{i−1}` y `r_i ≤ r_{i+1}`. Es el punto de arranque de un nuevo segmento lineal de la sierra de restos (dentro de un segmento con `q_i` constante, `r` decrece con pendiente −1).
- **B-liso:** entero cuyos factores primos son todos ≤ B.
- **SaltoMaximo(n):** el mayor salto del resto entre segmentos consecutivos.

Lema del Mínimo Liso

> **Lema.** Sea `n` compuesto y `r_i = n mod i`. Si `i` es punto de mínimo relativo de la secuencia de restos, entonces `r_i` es, con probabilidad muy superior a la de un resto arbitrario, **B-liso**; o bien `r_i = 0`, en cuyo caso **`i | n`** y se obtiene un factor directamente. Además, la secuencia `i` de mínimos es **simétrica respecto a `√n`**: por cada mínimo `(i, q, r)`, su espejo `q = ⌊n/i⌋` pertenece al lado opuesto de la hipérbola `q·i = n − r`.

Corolario (regla dura): en la criba de mínimos no hay grises: *o el resto es liso, o es el factor.*

Evidencia numérica (n = 1007 = 19·53, B = 20, i ≤ 60)

i	q = n//i	r = n%i	¿liso?
5	201	2	2
11	91	6	2·3
13	77	6	2·3
15	67	2	2
17	59	4	2^2
19	53	**0**	**FACTOR**
27	37	8	2^3
31	32	15	3·5

i	q = n//i	r = n%i	¿liso?
32	31	15	3·5
37	27	8	2^3
43	23	18	$2 \cdot 3^2$
47	21	20	$2^2 \cdot 5$
53	19	**0**	**FACTOR**
59	17	4	2^2

18/20 mínimos dan restos B-lisos; los 2 restantes son los factores exactos. Simetría visible: (27↔37), (31↔32), (19↔53) respecto a √1007 ≈ 31.7.

Validación automática del test propio (espejo del algoritmo, ejecutado por Lee):

- T1 lisidad de mínimos: **20/20** ▓
- T2 factores como r=0: **19 y 53** ▓
- T5 factorización determinista: **10/10 semiprimos** ▓

Conexión con Lehman y fracciones continuas

Los mínimos relativos de `n mod i` coinciden en estructura con los denominadores de los convergentes de la fracción continua de `√n`: ambos son los puntos donde el resto queda *mínimo* respecto a su entorno. La simetría especular es la misma que explota el método de Lehman (1974) para factorizar en O(n^(1/3)). El generador de gráficas del Libro 2 redescubre, de forma independiente, esta maquinaria.

Aplicación a GNFS: `gnfs_minimo.py`

La **GNFS-mini con oráculo de mínimos** integra el Lema en las 5 fases clásicas:

1. **Polinomio base-m:** $n = m^2 + c_1 m + c_0$, $f(x) = x^2 + c_1 x + c_0$, $f(m) \equiv 0 \pmod{n}$. El cuerpo es cuadrático imaginario ($D = c_1^2 - 4c_0 < 0$).
2. **Oráculo 33×1:** los candidatos (a, b) se generan **solo** en los mínimos relativos: $a = b \cdot m \pm i$ con i mínimo relativo de $n \bmod i$. El lado racional $R = a - b \cdot m = \pm i$ es exactamente el resto mínimo — pequeño por construcción, candidato liso por el Lema.
3. **Relaciones:** R B-lisa **y** $\mathrm{Norm}(a - b\theta) = a^2 + c_1 ab + c_0 b^2$ B-lisa. Columnas algebraicas por **par (p, raíz)** $r = a \cdot b^{-1} \bmod p$, no por primo.
4. **Álgebra lineal GF(2)** → dependencias.
5. **Raíz algebraica en el cuerpo cuadrático:** $\beta = x + y\theta$; como $\mathrm{Norm}(\beta) = v^2$ es cuadrado perfecto, $y = a + b\sqrt{D}$ con $a^2 = (\pm A' \pm 2v)/4$. Entonces $Y = \varphi(y) = a + b(2m + c_1) \pmod{n}$ y $u_0^2 = \prod R$ son **dos raíces cuadradas distintas** → $\gcd(Y - u_0, n)$ parte n.

Semiprimos crackeados por el pipeline completo (validado)

n	factores	relaciones	columnas	dependencias
100 160 063	10 007 × 10 009	500	399	112
62 773 913	7 919 × 7 927	261	252	15
982 634 309	31 337 × 31 357	1 040	620	422

Corolario 2v+3 — la grilla del Libro 5 podada por los mínimos

Identidad (corregida y verificada): $N = (2l+3)(2f+3) \iff v = (N-3)/2 = 2(l+3/2)(f+3/2) - 3/2$. Cada l genera el candidato impar $a = 2l+3$, con $l = 0..(\sqrt{N}-3)/2$ — la grilla completa es división por tentativa reparametrizada (verificado: en N = 100.160.063 solo funciona l = 5002 → a = 10 007).

> **Lema del Divisor Mínimo.** *Todo divisor* a *de* n *es un mínimo relativo de la secuencia* $r_i = n \bmod i$, *porque* $r_a = 0 \le r_{a-1}, r_{a+1}$. (Verificado: 24/24 semiprimos, cero violaciones.)

Consecuencia: podar la grilla 2v+3 a los mínimos impares **no pierde factores, garantizado**. Poda medida: sobrevive el **25–29%** de la grilla (4× menos candidatos, sin falsos negativos). La densidad de mínimos no es uniforme: ~26% global, cayendo a ~11% en el último decil (cerca de √n, los mínimos se espacian).

Como generador de relaciones GNFS, en cambio, el oráculo rinde ≈ muestreo aleatorio (14,6% vs 15,1% con B=1000): **la secuencia i de mínimos es peine de factores, no acelerador de criba** — dos oficios distintos.

Validación de `factor_2v3.py` (espejo ejecutado): 12/12 semiprimos de 6, 8 y 10 cifras factorizados; poda media 27,3% / 27,3% / 25,5%.

Archivos del paquete (33×1_criba_minimos_lisos.zip)

- `minimo_liso.py` — biblioteca: mínimos relativos, SaltoMaximo, factorización determinista, test de densidad.
- `gnfs_minimo.py` — **GNFS-mini con oráculo de mínimos**: ataca semiprimos con el pipeline completo.
- `test_lee.py` — suite de pruebas: *"Lee fluye por la red en busca del gran GNFS"* (T1–T6).
- `README.md` — este paper.
- `gen_zip.sh` — el propio empaquetador ZIP artesanal (sin herramientas externas), firma del estilo 33×1.

#33×1 · #nomaimai